

Bottling Line Data Capture

Packaging-Line Downtime Capture & Review System

Design & Testing Document

Ini Ukut

MSc Software Engineering — Capstone Project

Quantic School of Business and Technology

Contents

1. Introduction & Problem Statement	3
2. Requirements.....	3
3. System Architecture.....	4
4. Design Decisions & Patterns	5
5. Data Model	6
6. Technology Stack	7
7. Security & Audit	7
8. Testing Strategy	8
9. Test Results	9
10. Deployment.....	9
11. Limitations & Future Work.....	10
12. Conclusion.....	10

1. Introduction & Problem Statement

This document describes the design and testing of the Bottling Line Downtime Data Capture system, a web application that records and reviews production-line downtime for a beverage bottling operation. It accompanies the source repository and the recorded demonstration, and is intended to explain what was built, the engineering decisions behind it, and the evidence that it works.

1.1 The problem

On a packaging line, every stoppage at the filler, labeler or any other machines on the production line must be recorded with its time, duration, machine, product, and reason. In the current process this is captured on paper and in shared Excel workbooks, which produces recurring data-quality problems:

- Reasons for downtime are entered as free text, so the same fault is recorded inconsistently and cannot be aggregated or analyzed.
- Durations are typed by hand into several columns (minutes, hours), which frequently disagree with one another.
- Entries could miss and errors are only discovered days later, during reporting.
- There is no audit trail: no record of who entered a value, when, or whether it was reviewed.

The consequence is that supervisors spend significant time validating and reconciling data that should have been correct at the point of entry.

1.2 Objectives

- Capture downtime correctly at entry through validated forms and controlled vocabularies.
- Provide a supervisor review workflow that flags anomalies and produces an approved, immutable record.
- Maintain a full audit trail (who entered, who approved, and when).
- Continue to produce the legacy 18-column Excel workbook so downstream reporting is unaffected by migration.

2. Requirements

2.1 Functional requirements

ID	Requirement
FR-1	Users authenticate with a username and password; access is governed by role (operator, supervisor, admin).
FR-2	An operator can open a shift and record loss events using validated forms and dropdown lookups.
FR-3	The system derives all durations and calendar fields from the entered times; operators never type them.
FR-4	Selecting a Type of Loss suggests a standard BCS category, which the operator may override.
FR-5	An operator can close a shift, submitting it for supervisor review; it then becomes read-only.

ID	Requirement
FR-6	A supervisor sees a dashboard of shifts grouped by status with downtime totals and anomaly counts.
FR-7	The system automatically flags anomalies (long events, high total downtime, repeated machine failures).
FR-8	A supervisor can approve and then lock a shift, producing an immutable audited record.
FR-9	A supervisor can export a shift in the legacy 18-column Excel format.

2.2 Non-functional requirements

- Data integrity: each fact is stored once; derived values cannot diverge.
- Security: passwords hashed with bcrypt; sessions in signed cookies; role-enforced authorization.
- Maintainability: layered architecture with all database access isolated behind repositories.
- Portability: the database can move from SQLite (demo) to PostgreSQL (intranet) without application changes.
- Quality assurance: automated tests and continuous integration gate every change to the main branch.

3. System Architecture

The application follows three-tier layered architecture. Each incoming request passes through a route, which delegates business logic to a service, which reads and writes data exclusively through a repository. Templates render the response as server-side HTML.

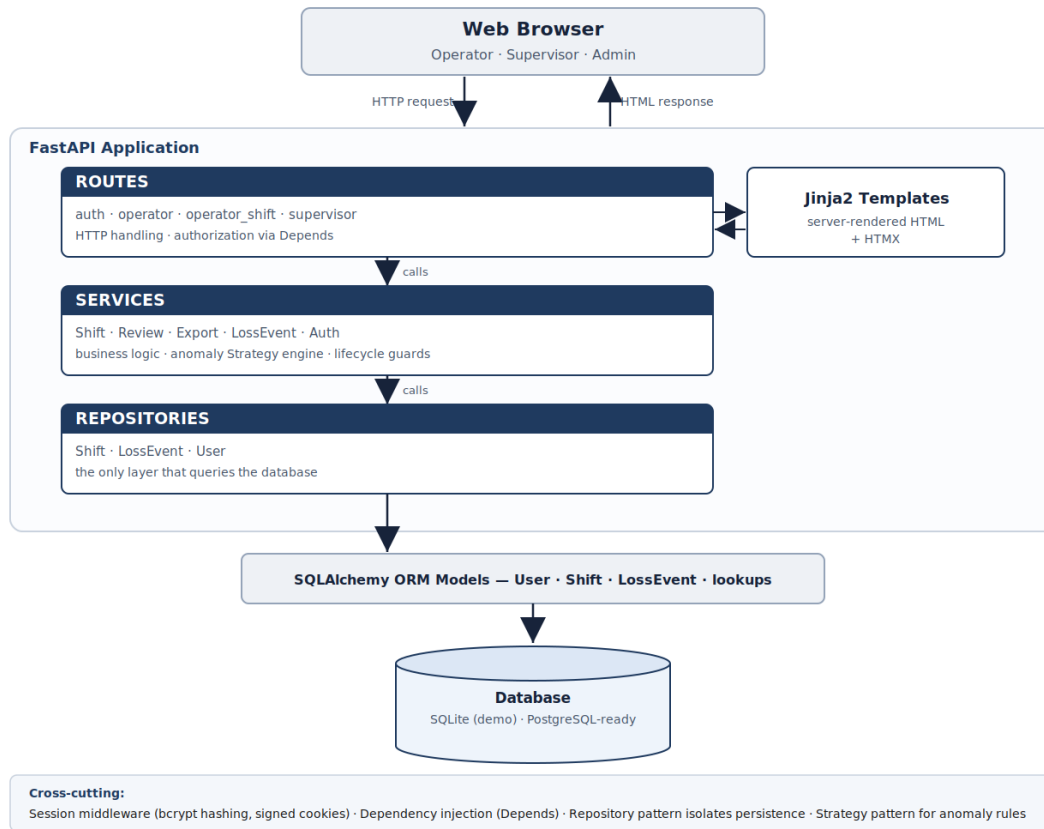


Figure 1. Layered architecture and request / response flow.

Layer	Responsibility	Examples
Routes	HTTP handling: parse requests, enforce auth, return pages/files.	auth, operator, operator_shift, supervisor
Services	Business logic: shift lifecycle, validation, anomaly detection, export.	ShiftService, ReviewService, ExportService, LossEventService
Repositories	Data access: the only code that queries the database.	ShiftRepository, LossEventRepository, UserRepository
Models	SQLAlchemy ORM entities mapped to tables.	User, Shift, LossEvent, lookups
Templates	Server-rendered Jinja2 views (with light HTMX).	operator/, supervisor/, auth/

This separation gives two concrete benefits. First, business rules are testable without a running web server, because they live in the service layer rather than inside request handlers. Second, because every database interaction is funnelled through repositories, the persistence technology can be changed in one place.

4. Design Decisions & Patterns

4.1 Derive, don't store (the core decision)

The database stores only the values an operator actually types: a date, a start time, a stop time, and a small set of dropdown selections. Every representation the legacy sheet stored redundantly the three

duration formats and five calendar fields — is computed on demand from those inputs. This eliminates the bug class where duration-in-minutes and duration-in-hours disagree, because each fact exists exactly once. The derivation logic is unit-tested against real historical April-2020 data to guarantee it reproduces the legacy figures.

4.2 Repository pattern

All persistence is encapsulated behind repository classes. Services depend on repositories, never on the ORM session directly, which keeps queries in one layer and makes a future SQLite-to-PostgreSQL migration change.

4.3 Dependency injection

FastAPI's Depends mechanism injects the database session and the authenticated user into routes. Authorisation is expressed declaratively via a `require_role(...)` dependency, so access rules are visible at the route definition and are trivially testable.

4.4 Strategy pattern for anomaly detection

Each anomaly check is an independent rule class implementing a common interface and registered in a single list that the review service iterates. Adding a rule is one new class plus one registry entry, with no change to existing rules — an application of the Open/Closed Principle.

4.5 Guided entry with override

Because a Type of Loss usually — but not always — maps to a fixed BCS category, the form pre-selects the standard category yet allows the operator to change it. This makes the common path fast while still permitting legitimate exceptions.

4.6 Shift lifecycle as a guarded state machine

A shift moves through OPEN → PENDING_REVIEW → APPROVED → LOCKED. Every transition is guarded in the service layer; illegal transitions raise a dedicated error rather than silently corrupting state. Once past OPEN, a shift is read-only to operators; once LOCKED, it is immutable without an admin override.

5. Data Model

The schema was derived from the structure of the real brewery workbook. A central loss event table stores typed values only; four lookup tables provide controlled vocabularies; and the user and shift tables carry authentication and lifecycle state.

Table	Purpose	Key columns
user	Accounts and roles.	username, password_hash, role, is_active
shift	A production shift and its lifecycle.	shift_date, shift_name, line, operator_id, supervisor_id, status, opened_at, closed_at, approved_at
loss event	A single downtime event (typed values only).	event_date, event_start, event_stop, sku_id, machine_id, loss_type_id, bcs_category_id, failure_descriptions, created_by_id

Table	Purpose	Key columns
sku	Product lookup.	code
machine	Machine lookup.	name
bcs_category	Five locked standard categories.	code
loss_type	Operational loss types with a suggested BCS.	code, suggested_bcs_code

5.1 Derived fields

Computed from event_date, event_start, and event_stop: Duration (h:mm), Duration (min), Duration (hr), Month, Week (ISO), Year, Week-Year, and Month-Year.

5.2 Shift lifecycle states

OPEN (operator editing) → PENDING_REVIEW (submitted, read-only) → APPROVED (official record) → LOCKED (immutable; admin override required to change).

6. Technology Stack

Component	Technology	Role
Web framework	FastAPI	Routing, dependency injection, request validation
ORM / database	SQLAlchemy 2.0 + SQLite	Persistence (PostgreSQL-ready)
Validation	Pydantic 2	Typed request/settings models
Templating	Jinja2 + HTMX	Server-rendered views with light interactivity
Auth	bcrypt + itsdangerous	Password hashing and signed session cookies
Excel export	openpyxl	Legacy 18-column workbook generation
Testing	pytest + coverage	Automated unit and integration tests
Quality	ruff	Linting and formatting
CI/CD	GitHub Actions + Render	Automated checks and deployment

7. Security & Audit

- Passwords are never stored in plaintext; they are hashed with bcrypt (salted, adaptive cost).
- Sessions are held in signed cookies; only a user id is stored, and the user is re-loaded and re-checked on every request, so a deactivated account loses access immediately.
- Authorization is role-based and enforced server-side; the login flow returns an identical message for unknown-user and wrong-password to prevent username enumeration.
- The post-login redirect is sanitized to same-site paths only, guarding against open-redirect abuse.
- Approved and locked shifts are immutable, and each carries the operator, supervisor, and timestamps that form the audit trail.

8. Testing Strategy

Testing follows the test pyramid: a broad base of fast unit tests over the service and derivation logic, and a focused layer of integration tests that drive the real HTTP routes end to end. The suite currently comprises 95 automated tests.

8.1 Test breakdown

Level	Test module	Tests	Focus
Unit	test_derived_and_anomaly	14	Derived fields vs. real 2020 data; anomaly rules
Unit	test_loss_classification	7	Guided BCS suggestion logic
Unit	test_auth_service	10	Password hashing/verification; role guards
Unit	test_review_service	10	Lifecycle transitions; anomaly aggregation; dashboard
Unit	test_export_service	6	18-column layout and derived values
Integration	test_auth	12	Login, logout, guards, role enforcement
Integration	test_supervisor	11	Dashboard, review, approve/lock, operator close
Integration	test_export	5	Export route: auth, content type, workbook contents
Integration	test_start_shift / test_operator_form	11	Shift open and loss-event entry flows
Integration	test_app_smoke / test_seed	9	Application boot and seed integrity
	Total	95	

8.2 Anomaly rules under test

Rule	Trigger	Severity
LongBreakdownRule	A single event longer than 30 minutes	Warning; Critical if over 60 minutes
HighTotalDowntimeRule	Total shift downtime over 120 minutes	Warning; Critical if over 240 minutes
RepeatedMachineFailureRule	The same machine failing 3+ times in a shift	Warning

8.3 Continuous integration

A GitHub Actions workflow runs on every pull request against the protected main branch. It executes three gates on both Python 3.11 and 3.12:

- ruff check — linting;
- ruff format --check — formatting conformance;
- pytest with coverage — the full test suite, with a coverage report produced on Python 3.11.

The main branch requires pull requests and a passing CI run before merge; direct pushes are blocked. Each feature was delivered as its own reviewed, green pull request, and the course grader is a collaborator on the repository with full visibility of this history.

8.4 Requirements traceability

Every functional requirement is exercised by at least one automated test. The matrix below maps each requirement to the test modules and representative cases that verify it.

ID	Requirement	Verifying tests (module — representative cases)
FR-1	Authentication & role-based access	test_auth — login success/failure, route guards, role enforcement; test_auth_service — password hashing/verification, require_role
FR-2	Open a shift & record loss events	test_start_shift — open/resume a shift; test_operator_form — event entry and input validation
FR-3	Durations & calendar fields derived, not typed	test_derived_and_anomaly — derived values asserted against real April-2020 data
FR-4	Guided BCS suggestion with override	test_loss_classification — suggestion logic; test_operator_form — suggested-BCS partial and override
FR-5	Close shift → pending review, read-only	test_review_service — submit_for_review transition; test_supervisor — operator close, read-only entry
FR-6	Supervisor dashboard grouped by status	test_review_service — dashboard grouping, totals, anomaly counts; test_supervisor — dashboard renders
FR-7	Automatic anomaly flagging	test_derived_and_anomaly — rule triggers; test_review_service — aggregation; test_supervisor — anomaly shown on review
FR-8	Approve then lock; immutable audited record	test_review_service — approve/lock transitions and guards; test_supervisor — approve-then-lock flow, invalid transitions
FR-9	Legacy 18-column Excel export	test_export_service — 18-column layout and derived values; test_export — route auth, content type, workbook contents

9. Test Results

All 95 tests pass, and all three CI gates (lint, format, tests) are green on both supported Python versions. The suite grew alongside the feature set — 41 tests after the operator-entry milestone, 63 after authentication and role-based access, 84 after the supervisor review workflow, and 95 after the legacy export — with every increment merged only on a green pipeline.

The derived-field tests are the strongest single piece of evidence for correctness: by asserting actual values from the historical workbook, they demonstrate that the new system reproduces the legacy calculated columns exactly, which is what justifies trusting the Excel export without manual re-checking.

10. Deployment

The application is deployed to Render and rebuilt automatically when main updates. The public demo uses SQLite on the free tier, whose storage is ephemeral: seed data (users and lookups) is recreated on each deploy, while shift data entered during a session does not persist across redeployments. The production

target is the brewery intranet with a persistent PostgreSQL database — a change the repository layer makes straightforward. Demo accounts use username-equals-password credentials strictly for evaluation convenience and are not representative of production authentication.

11. Limitations & Future Work

- Admin user management (create/deactivate accounts, reset passwords) through the UI.
- A supervisor analytics view for downtime trends across shifts and machines (toward OEE reporting).
- A date-range exports to produce a monthly legacy workbook in a single operation.
- Migration of ORM timestamp defaults to time zone-aware datetimes, removing a deprecation warning.
- Persistent storage on the production deployment to retain entered data across restarts.
- Notification for a supervisor to the operator's works is yet to be included.

12. Conclusion

The project replaces a manual, error-prone, spreadsheet-based process with a validated, reviewed, and audited system, while preserving the exact legacy Excel output that downstream reporting depends on. Its layered architecture, deliberate design decisions — chiefly deriving rather than storing computed values — and a disciplined, test-and-CI-gated delivery process together produce software that is correct, maintainable, and ready to move from demonstration to a persistent production deployment.